

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Practical :- VI

Roll No :- T053	Name :- Nidhi Yadav
Class :- TYIT	Batch :- 2
Date of Assignment :- 19/08/2026	Date/Time of Submission :- 119/08/2026

Probability Reasoning (Bayesian Networks & Inference)

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code:-

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

✓ 0.0s

```
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
```

✓ 0.0s

```
# P(Burglary)
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)

# P(Earthquake)
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)

# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
```

```
# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # JohnCalls = False
        [0.05, 0.90] # JohnCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)

# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30], # MaryCalls = False
        [0.01, 0.70] # MaryCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "MaryCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
```

```
# Step 3: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)
inference = VariableElimination(model)

result = inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)

print("\nProbability of Burglary given JohnCalls=True and MaryCalls=True:")
print(result)
print("Nidhi Yadav T053")
```

Probability of Burglary given JohnCalls=True and MaryCalls=True:

Burglary	phi(Burglary)
Burglary(False)	0.7158
Burglary(True)	0.2842

Nidhi Yadav T053

2. Write a program to compute Posterior Probability using Bayesian Networks(Burglar|Alarm = True)

Code:-

```
# Posterior Probability using Bayesian Network
# Query: P(Burglary | Alarm = True)
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm")
])
# Step 2: Define probability of Burglary

cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
# Step 3: Define probability of Earthquake

cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
```

```
# Step 4: Define probability of Alarm
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
# Step 5: Add CPDs to model

model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm
)
```

```
# Step 6: Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())
# Step 7: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)
print("\nPosterior Probability:")
print("P(Burglary | Alarm = True)")
print(result)
print("Nidhi Yadav T053")
```

Is Bayesian Network valid? True

Posterior Probability:
P(Burglary | Alarm = True)

Burglary	phi(Burglary)
Burglary(False)	0.6264
Burglary(True)	0.3736

Nidhi Yadav T053

PRACTICAL 6A EXAMPLE (Hospital Emergency Alert System)

```

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Step 1: Create Bayesian Network
model = DiscreteBayesianNetwork([
    ("SeriousCondition", "EmergencyAlarm"),
    ("HighFever", "EmergencyAlarm"),
    ("EmergencyAlarm", "DoctorCalls"),
    ("EmergencyAlarm", "NurseCalls")
])

# P(SeriousCondition)
cpd_serious = TabularCPD(
    variable="SeriousCondition",
    variable_card=2,
    values=[[0.90], [0.10]],
    state_names={"SeriousCondition": ["False", "True"]}
)

# P(HighFever)
cpd_fever = TabularCPD(
    variable="HighFever",
    variable_card=2,
    values=[[0.80], [0.20]],
    state_names={"HighFever": ["False", "True"]}
)

```

```

# P(EmergencyAlarm | SeriousCondition, HighFever)
cpd_alarm = TabularCPD(
    variable="EmergencyAlarm",
    variable_card=2,
    values=[
        [0.99, 0.70, 0.60, 0.10], # EmergencyAlarm = False
        [0.01, 0.30, 0.40, 0.90] # EmergencyAlarm = True
    ],
    evidence=["SeriousCondition", "HighFever"],
    evidence_card=[2, 2],
    state_names={
        "EmergencyAlarm": ["False", "True"],
        "SeriousCondition": ["False", "True"],
        "HighFever": ["False", "True"]
    }
)

# P(DoctorCalls | EmergencyAlarm)
cpd_doctor = TabularCPD(
    variable="DoctorCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # DoctorCalls = False
        [0.05, 0.90] # DoctorCalls = True
    ],
    evidence=["EmergencyAlarm"],
    evidence_card=[2],
    state_names={
        "DoctorCalls": ["False", "True"],
        "EmergencyAlarm": ["False", "True"]
    }
)

```

```

# P(NurseCalls | EmergencyAlarm)
cpd_nurse = TabularCPD(
    variable="NurseCalls",
    variable_card=2,
    values=[
        [0.98, 0.20], # NurseCalls = False
        [0.02, 0.80] # NurseCalls = True
    ],
    evidence=["EmergencyAlarm"],
    evidence_card=[2],
    state_names={
        "NurseCalls": ["False", "True"],
        "EmergencyAlarm": ["False", "True"]
    }
)

# Step 2: Add CPDs to model
model.add_cpds(
    cpd_serious,
    cpd_fever,
    cpd_alarm,
    cpd_doctor,
    cpd_nurse
)

```

```

# Step 3: Perform inference
inference = VariableElimination(model)

result = inference.query(
    variables=["SeriousCondition"],
    evidence={
        "DoctorCalls": "True",
        "NurseCalls": "True"
    }
)

# Step 4: Display result
print("\nProbability of Serious Patient Condition")
print("given DoctorCalls=True and NurseCalls=True:")
print(result)

print("Nidhi Yadav T053")
0.0s

```

Probability of Serious Patient Condition
given DoctorCalls=True and NurseCalls=True:

SeriousCondition	phi(SeriousCondition)
SeriousCondition(False)	0.5547
SeriousCondition(True)	0.4453

Nidhi Yadav T053

PRACTICAL 6 B EXAMPLE (Hospital Emergency Alert System)

```
# Posterior Probability using Bayesian Network
# Query: P(SeriousCondition | EmergencyAlarm = True)
```

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

```
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("SeriousCondition", "EmergencyAlarm"),
    ("HighFever", "EmergencyAlarm")
])
```

```
# Step 2: Define probability of Serious Condition
```

```
cpd_serious = TabularCPD(
    variable="SeriousCondition",
    variable_card=2,
    values=[[0.90], [0.10]],
    state_names={"SeriousCondition": ["False", "True"]}
)
```

```
# Step 3: Define probability of High Fever
```

```
cpd_fever = TabularCPD(
    variable="HighFever",
    variable_card=2,
    values=[[0.80], [0.20]],
    state_names={"HighFever": ["False", "True"]}
)
```

```
# Step 4: Define probability of Emergency Alarm
# P(EmergencyAlarm | SeriousCondition, HighFever)
```

```
cpd_alarm = TabularCPD(
    variable="EmergencyAlarm",
    variable_card=2,
    values=[
        [0.99, 0.70, 0.60, 0.10], # EmergencyAlarm = False
        [0.01, 0.30, 0.40, 0.90] # EmergencyAlarm = True
    ],
    evidence=["SeriousCondition", "HighFever"],
    evidence_card=[2, 2],
    state_names={
        "EmergencyAlarm": ["False", "True"],
        "SeriousCondition": ["False", "True"],
        "HighFever": ["False", "True"]
    }
)
```

```
# Step 5: Add CPDs to model
```

```
model.add_cpds(
    cpd_serious,
    cpd_fever,
    cpd_alarm
)
```

```
# Step 6: Check whether model is correct
```

```
print("Is Bayesian Network valid?", model.check_model())
```

```
# Step 7: Perform inference
```

```
inference = VariableElimination(model)

result = inference.query(
    variables=["SeriousCondition"],
    evidence={"EmergencyAlarm": "True"}
)
```

```
# Step 8: Display result
```

```
print("\nPosterior Probability:")
print("P(SeriousCondition | EmergencyAlarm = True)")
print(result)

print("Nidhi Yadav T053")
```

✓ 0.0s

```
Is Bayesian Network valid? True
```

```
Posterior Probability:
P(SeriousCondition | EmergencyAlarm = True)
```

SeriousCondition	phi(SeriousCondition)
SeriousCondition(False)	0.5504
SeriousCondition(True)	0.4496

Nidhi Yadav T053